Practical course

Systems Programming

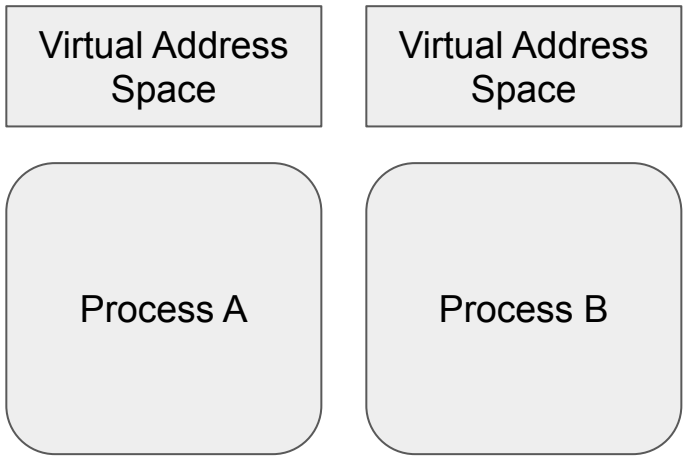
Systems Research Group

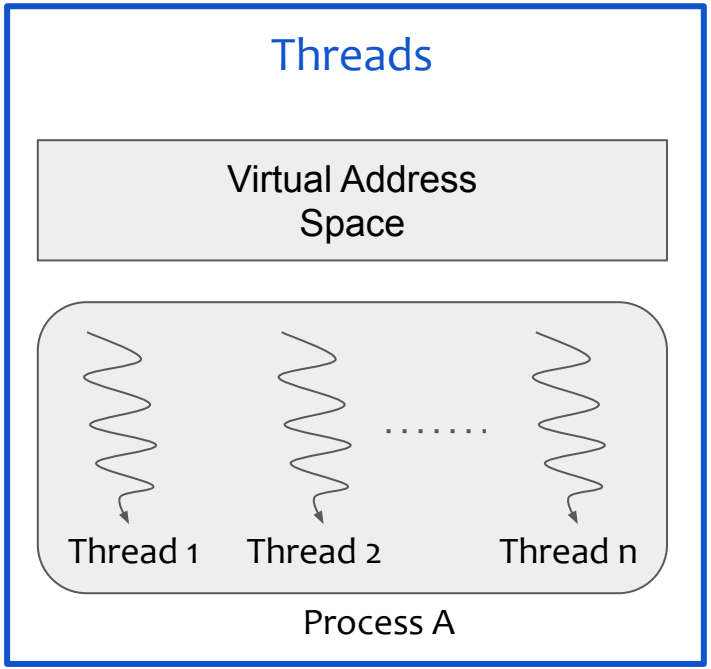
Today's topic!

Concurrency and Synchronization

Processes vs Threads

- Last lecture: inter-process parallelism
- Today: intra-process parallelism

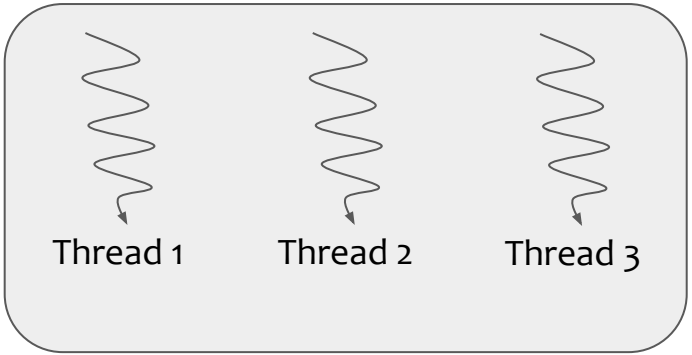




Threads

- Threads divide a program into independent units of computation
- Each thread has:
  - Program counter
  - Thread id
  - Stack
  - Set of registers
- Created using the `clone` system call

GUI	Physics Engine	Audio
Stack 1	Stack 2	Stack 3
Registers 1	Registers 2	Registers 3



POSIX Threads API

- Create a thread – `pthread_create`
- Wait for a thread to finish – `pthread_join`

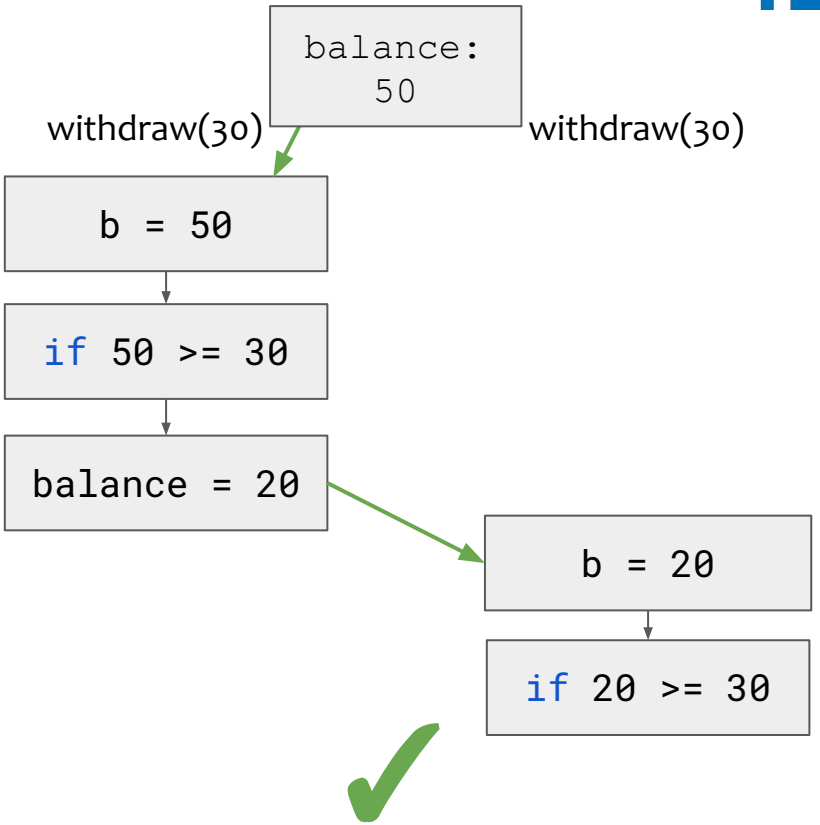
Example

```
#include <stdio.h>
#include <pthread.h>

void *thread(void *ptr) {
  int id = * (int *) ptr;
  printf( "Thread %d\n", id );
  return NULL;
}

int main( int argc, char **argv ) {
  pthread_t t1, t2;
  int id1 = 1; int id2 = 2;
  pthread_create( &t1, NULL, *thread, (void *) &id1 );
  pthread_create( &t2, NULL, *thread, (void *) &id2 );
  pthread_join( t1, NULL );
  pthread_join( t2, NULL );
  return 0;
}
```

The Need for Synchronization

- Threads share the same resources
  - Heap memory
  - Files descriptors, and other I/O
- Concurrent resource accesses affect domain logic

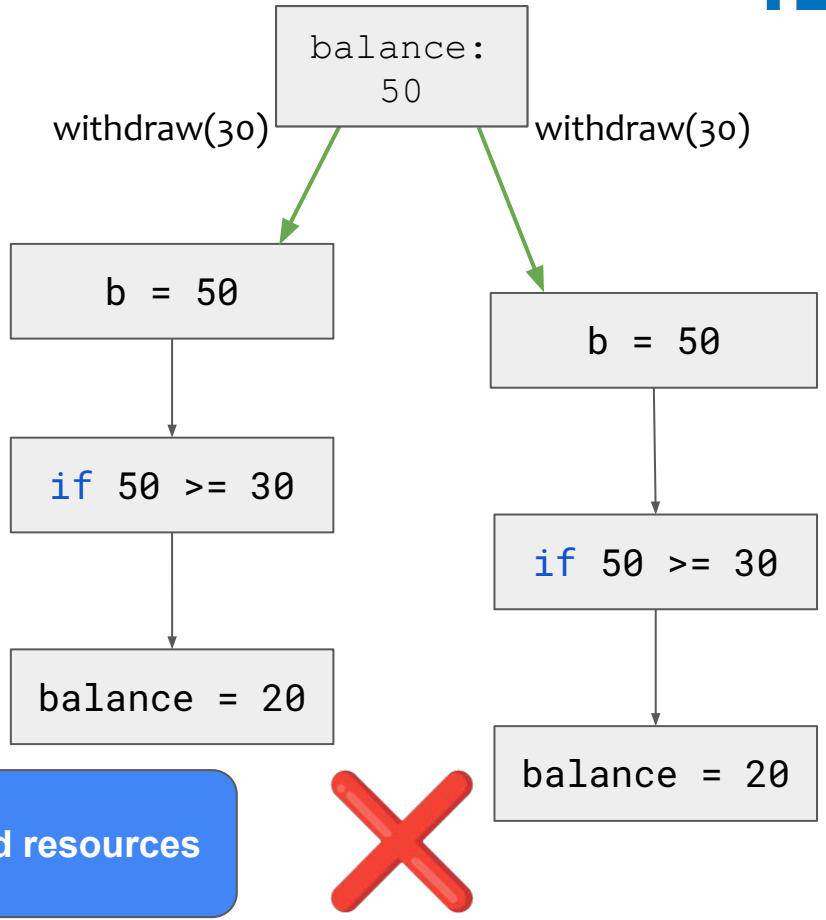
```
withdraw (amount) {
  b = balance
  if ( b >= amount )
     balance = b - amount
}
```



The Need for Synchronization

- Threads share the same resources
  - Heap memory
  - Files descriptors, and other I/O
- Concurrent resource accesses affect domain logic

```
withdraw (amount) {
  b = balance
  if ( b >= amount )
     balance = b - amount
}
```



Must consider simultaneous accesses to shared resources

Synchronization

- Various mechanisms for synchronization
  - Mutual exclusion
  - Lock-free
  - Read-Copy-Update
  - Transactional memory

Synchronization

- Various mechanisms for synchronization
  - **Mutual exclusion**
  - **Lock-free**
  - Read-Copy-Update
  - Transactional memory

Mutual Exclusion

Mutual Exclusion

- To grant a thread exclusive access to a shared resource
- Mutex or lock
  - Can be held by one thread at a time
  - Each thread must *acquire* the mutex *before* accessing the shared resource
  - Each thread must *release* the mutex *after* accessing the shared resource
- *Critical sections* will be executed *atomically*

```
withdraw (amount) {
  acquire_mutex()
  b = balance
  if ( b >= amount )
     balance = b - amount
  release_mutex()
}
```

Types of Mutexes

- Mutex
  - Allows at most one thread
- Semaphore
  - Can allow multiple threads at a time
  - They are usually used for signalling
- Read-writer locks
  - Allows readers to execute concurrently, but only one writer a time
    - e.g.: `check_balance` accesses a shared resource but does not modify it

Implementing Locks

- There is a plethora of algorithms for implementing locks
  - Test-and-set, Test-test-and-set, MCS, ticket locks, etc,
- Hardware atomic primitives
  - Read-Modify-Write
  - Atomic-Increment-Fetch

```
read_modify_write(int *var, int old_value, int new_value){
    int cur_val = *var;
    if (cur_val == old_value){
        *var = new_value;
    }
    return cur_val;
}
```

Atomic

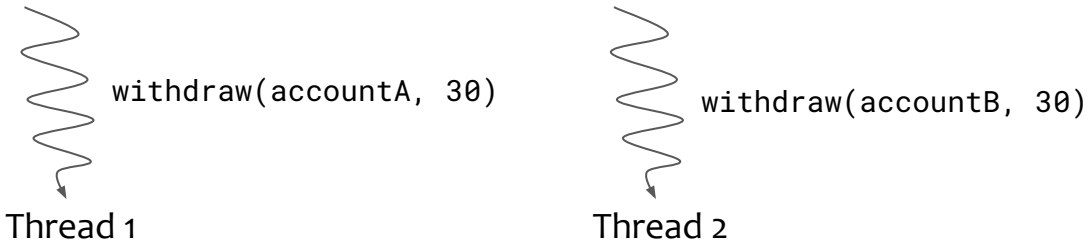
```
acquire_mutex(){
    while( read_modify_write(&mutex, 0, 1) != 0) ;
}
```

pthread Synchronization

- pthreads library has several mutexes:
  - `pthread_mutex`
    - threads enter sleep mode when lock is busy
    - `pthread_mutex_lock`, `pthread_mutex_unlock`, `pthread_mutex_trylock`
  - `pthread_spinlock`
    - threads keep polling the status of the lock
    - `pthread_spin_lock`, `pthread_spin_unlock`, `pthread_spin_trylock`
  - `pthread_rwlock`
    - `pthread_rwlock_rdlock`, `pthread_rwlock_wrlock`
  - sem
    - `sem_post`, `sem_wait`
- Other synchronization mechanisms:
  - `pthread_cond`
  - `pthread_barrier`

What about Performance?

```
withdraw (account, amount) {
  acquire_mutex()
  if ( account.balance >= amount )
     account.balance -= amount
  release_mutex()
}
```



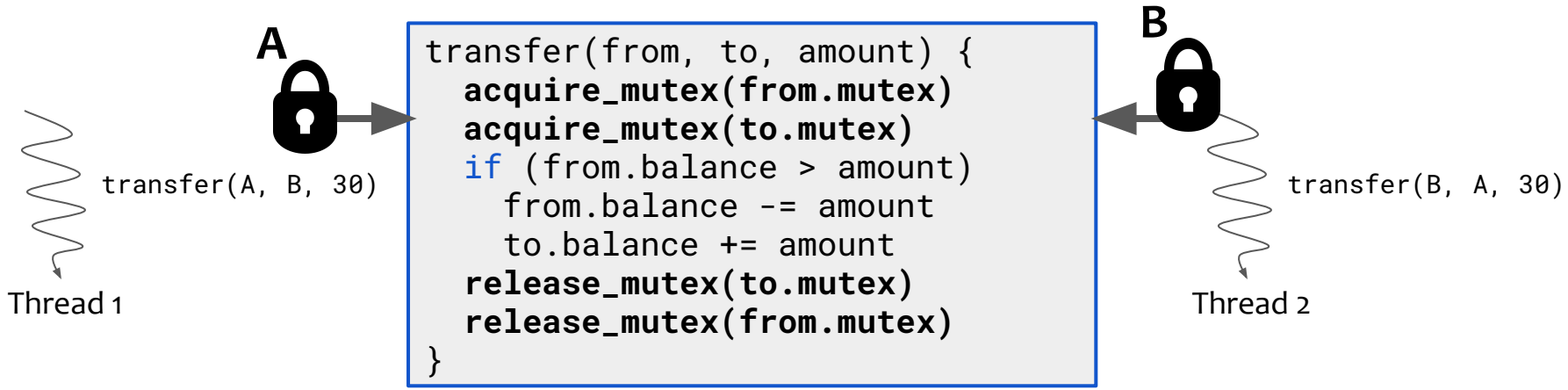
Synchronization can hamper scalability and performance

Scalable Synchronization

- Coarse-grained vs fine-grained locking
  - single global lock vs per-account lock
- Can allow higher parallelism
- Introduces memory overhead
  - Lock per array entry
- Challenging to design

```
withdraw (account, amount) {
  acquire_mutex( account.mutex )
  if ( account.balance >= amount )
     account.balance -= amount
  release_mutex( account.mutex )
}
```

Scalable Synchronization




```
transfer(from, to, amount) {
  acquire_mutex(from.mutex)
  acquire_mutex(to.mutex)
  if (from.balance > amount)
    from.balance -= amount
    to.balance += amount
  release_mutex(to.mutex)
  release_mutex(from.mutex)
}
```

```
acquire_mutex(A.mutex)
acquire_mutex(B.mutex)
```

```
acquire_mutex(B.mutex)
acquire_mutex(A.mutex)
```

Deadlock

Scalable Synchronization

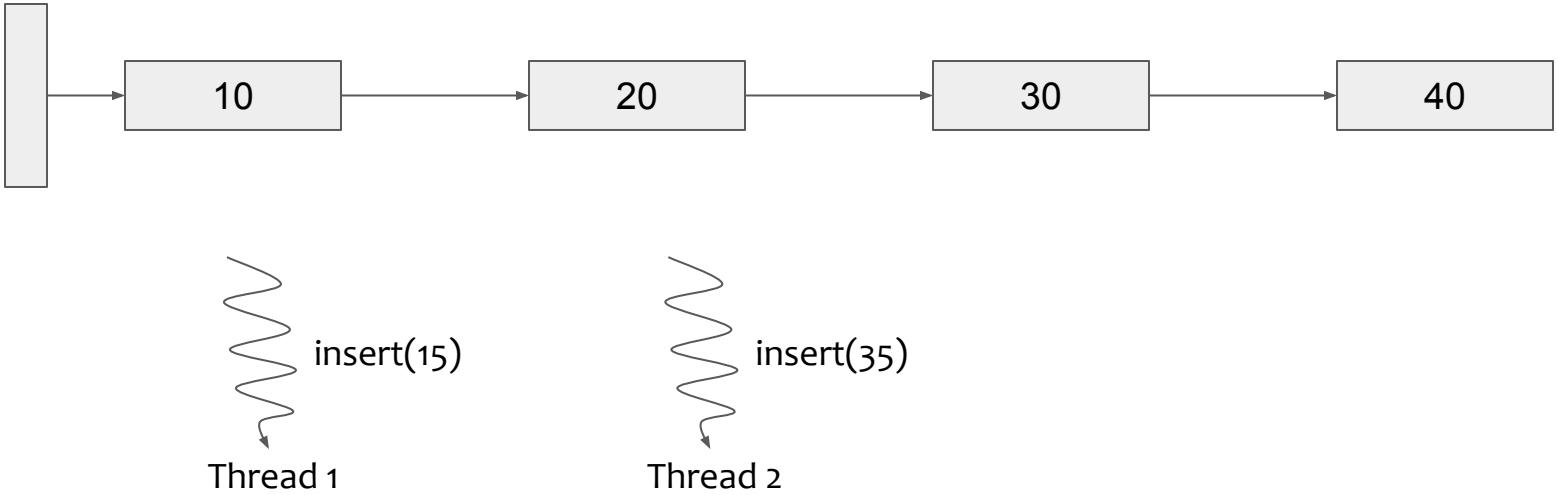
- Coarse-grained vs. fine-grained locking
  - single global lock vs. per-account lock
- Can allow higher parallelism
- Introduces memory overhead
  - e.g.: Lock per array entry
- Challenging to design
  - Deadlocks: threads cannot make progress
  - Livelocks: threads cannot make useful progress
  - Fairness and starvation

```
withdraw (account, amount) {
  acquire_mutex( account.mutex )
  if ( account.balance >= amount )
     account.balance -= amount
  release_mutex( account.mutex )
}
```

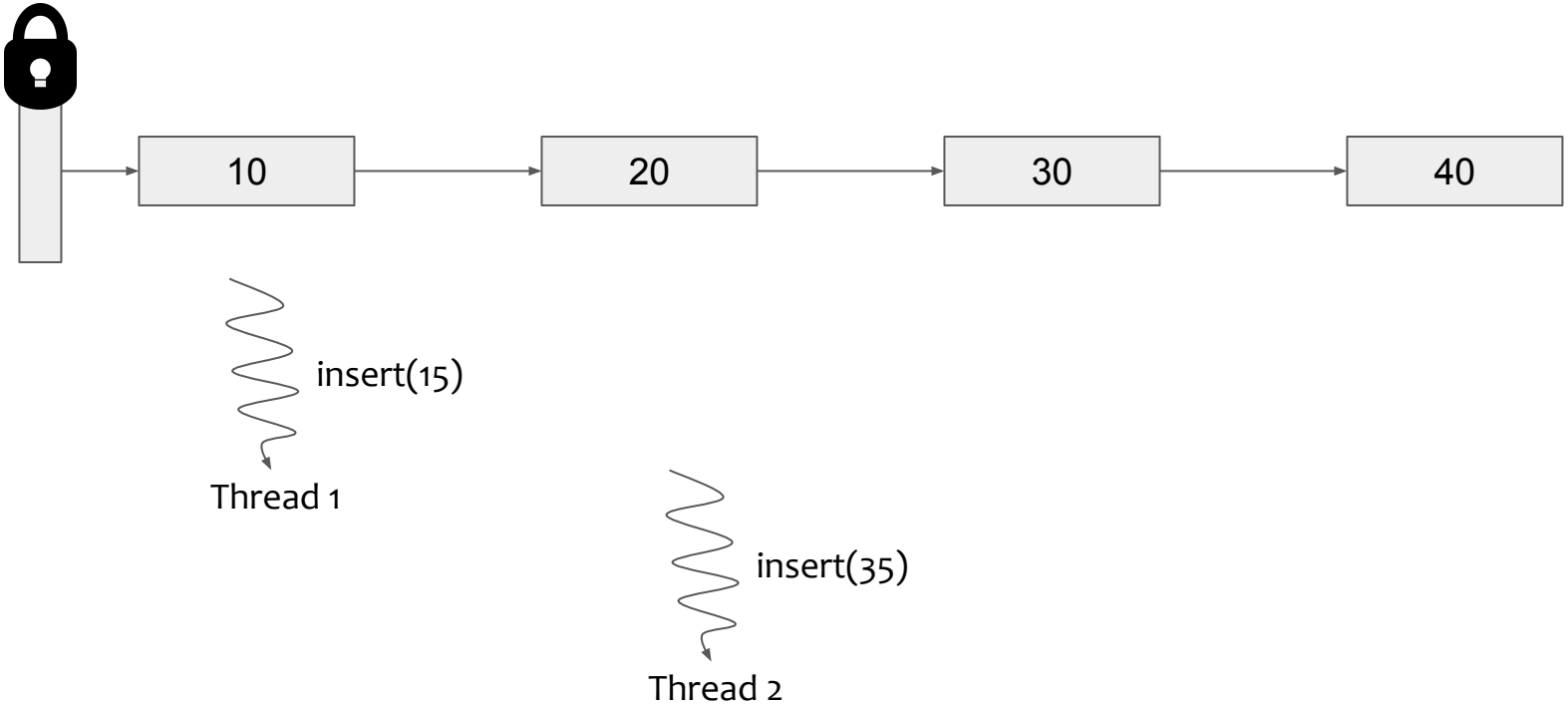
Lock-free Programming

Lock-free Programming

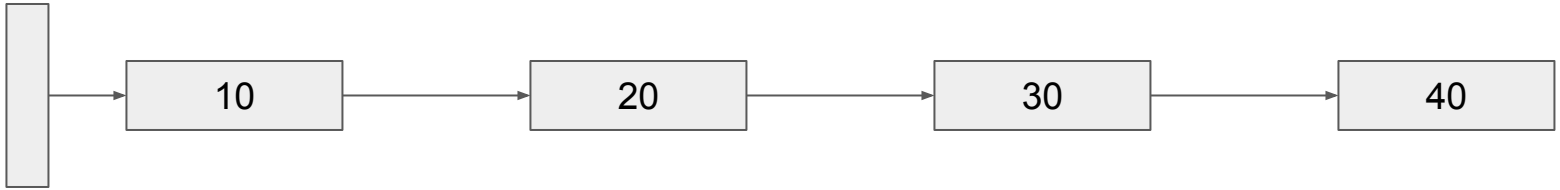
- Avoid using locks and rely on atomic primitives
  - Threads do not have to wait for a lock to be free



Lock-free Programming

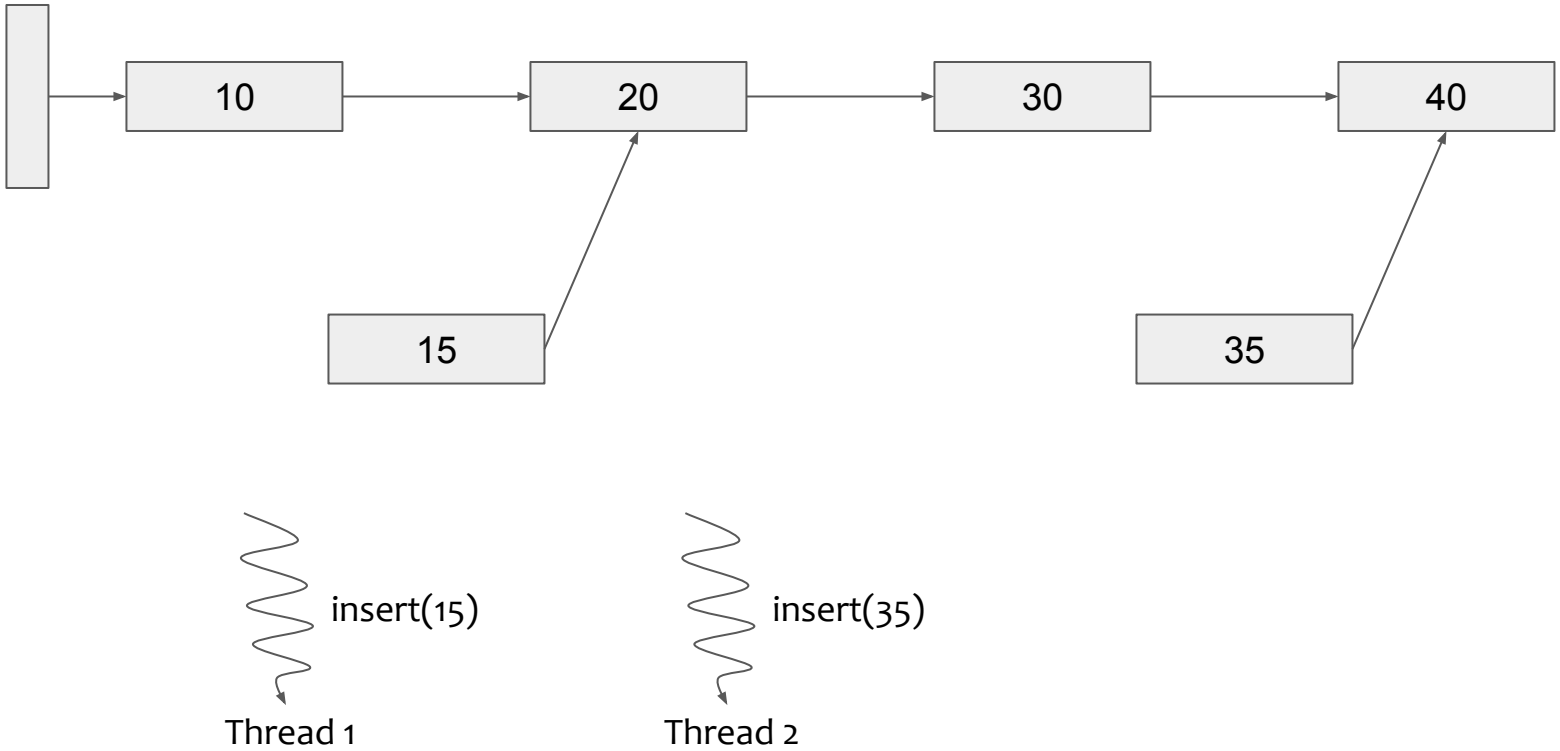


Lock-free Programming

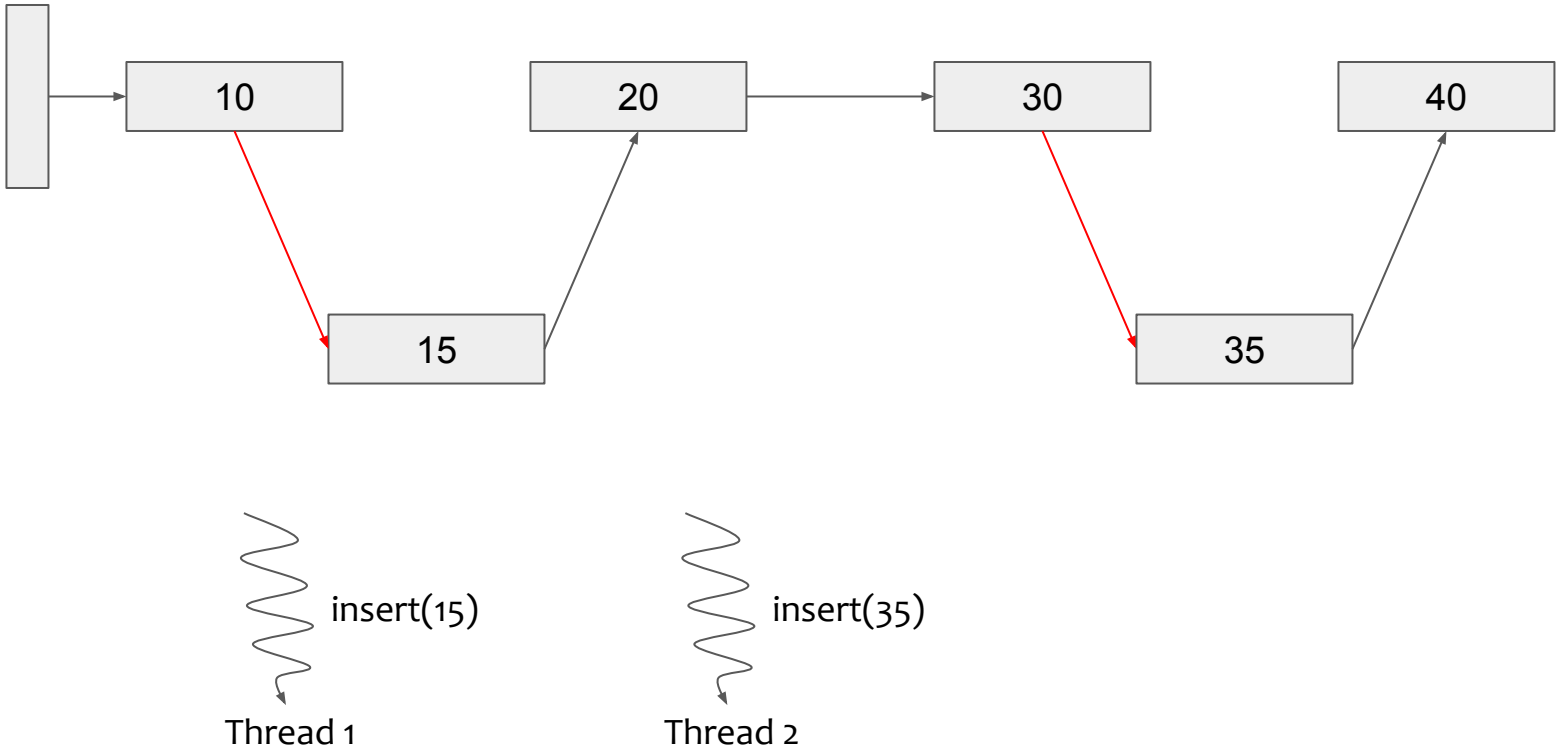



```
insert(value) {
    new_node.value = value
    node = find_prev(value)
    new_node.next = node.next
    read_modify_write(&node.next, node.next, new_node);
}
```

Lock-free Programming



Lock-free Programming



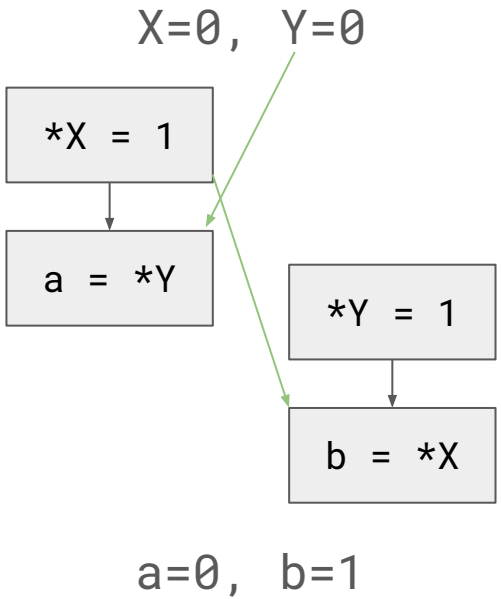
Challenges of Lock-free Programming

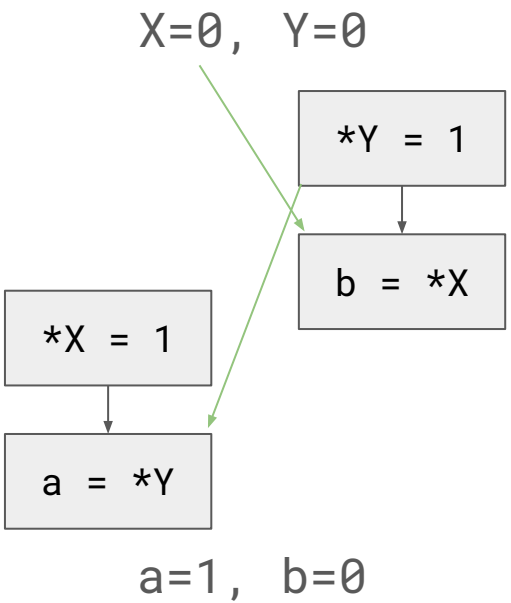
- Similar challenges to fine-grained locking
  - Starvation
  - Livelocks
- Memory reclamation
  - Threads cannot tell if other threads are referencing some data
  - When is it safe to free an object?

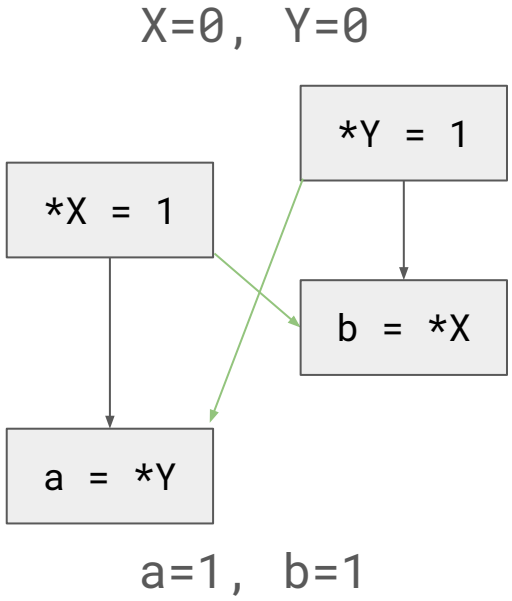
Weak Memory

Weak Memory

```
X=0, Y=0
*X = 1;  ||  *Y = 1;
a = *Y;  ||  b = *X;
```

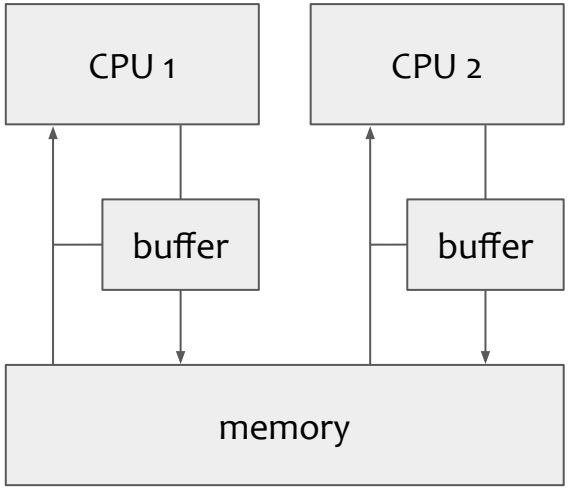


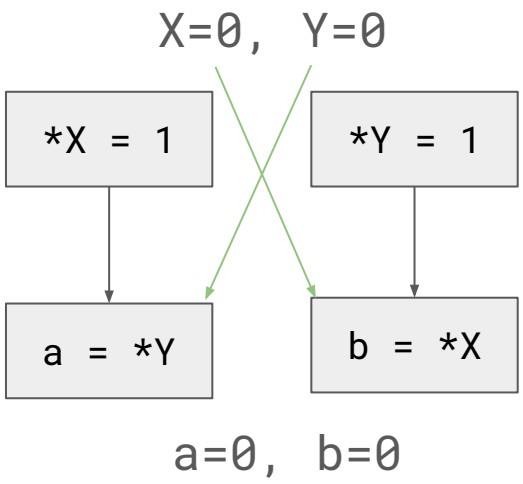




Weak Memory

```
X=0, Y=0
*X = 1;  ||  *Y = 1;
a = *Y;  ||  b = *X;
```





fence

```
#include
<stdatomic.h>

atomic_thread_fence(

memory_order_seq_cst
);
```

Conclusion

- Threads allow for lightweight intra-process concurrency
- Synchronization is needed to ensure program correctness
- Hardware atomic primitives allow for implementing efficient synchronization mechanisms
- Scalable synchronization is challenging
- Lock-free algorithms can further increase performance
- Memory ordering primitives reduce out-of-order execution

Thank you for listening!

see you in the Q&A session